

100,000 string objects, but it only performs one string concatenation, when it calls `join()`. For example, the following code produces the equivalent `finalString` but without the intermediate string concatenations:

```
>>> finalString = []
>>> for i in range(100000):
...     finalString.append('spam ')
...
>>> finalString = ''.join(finalString)
>>> finalString
spam spam spam spam spam spam spam spam spam spam --snip--
```

When I measure the runtime of these two pieces of code on my machine, the list appending approach is *10 times faster* than the string concatenation approach. (Chapter 13 describes how to measure how fast your programs run.) This difference becomes greater the more iterations the `for` loop makes. But when you change `range(100000)` to `range(100)`, although concatenation remains slower than list appending, the speed difference is negligible. You don't need to obsessively avoid string concatenation, f-strings, the `format()` string method, or `%s` format specifiers in every case. The speed only significantly improves when you're performing large numbers of string concatenations.

Python frees you from having to think about many underlying details. This allows programmers to write software quickly, and as mentioned earlier, programmer time is more valuable than CPU time. But there are cases when it's good to understand details, such as the difference between immutable strings and mutable lists, to avoid tripping on a gotcha, like building strings through concatenation.

Don't Expect `sort()` to Sort Alphabetically

Understanding sorting algorithms—algorithms that systematically arrange values by some established order—is an important foundation for a computer science education. But this isn't a computer science book; we don't need to know these algorithms, because we can just call Python's `sort()` method. However, you'll notice that `sort()` has some odd sorting behavior that puts a capital Z before a lowercase *a*: